

Sinhgad College of Engineering, Pune - 41

Department of Electronics & Telecommunication Engineering

T.E. (E&TC) – 2019 Pattern | Subject: Embedded Processors

ORAL EXAMINATION – QUESTIONS & ANSWERS

(All 10 Experiments with Detailed Answers)

Practical 1: Interfacing LPC2148 with Keyboard, LCD and Display Key Press on LCD

Q1. What is LPC2148? Which family does it belong to and what is its core architecture?

Ans: LPC2148 is a 32-bit microcontroller from NXP (Philips) based on the ARM7TDMI-S core. It belongs to the LPC2100 family. It runs at up to 60 MHz, has 512 KB Flash, 32+8 KB RAM, and supports multiple communication interfaces like UART, SPI, I2C, and USB.

Q2. What is the operating voltage and clock frequency of LPC2148?

Ans: LPC2148 operates at 3.3 V I/O voltage and the core supply is 1.8 V. The maximum CPU clock frequency is 60 MHz. An external crystal (typically 12 MHz) is used with an on-chip PLL to generate the desired CPU frequency.

Q3. How does a 4x4 matrix keyboard work? Explain row-column scanning.

Ans: In a 4x4 matrix keyboard, rows are driven as outputs and columns are read as inputs. Each row is pulled LOW one at a time. When a key is pressed, it connects a row to a column, making that column read LOW. The combination of the active row and the LOW column identifies the pressed key.

Q4. What are the different types of LCDs? Which one is commonly used?

Ans: Common LCD types: Character LCD (e.g., 16x2, 20x4), Graphical LCD (GLCD), TFT LCD. For LPC2148 practicals, the 16x2 character LCD (HD44780 controller) is most commonly used. It displays 16 characters per row across 2 rows.

Q5. Explain the difference between 4-bit and 8-bit mode of LCD interfacing.

Ans: In 8-bit mode, all 8 data lines (D0–D7) are connected, sending data in one operation. In 4-bit mode, only D4–D7 are connected, so data is sent in two nibbles (upper nibble first, then lower nibble). 4-bit mode uses fewer GPIO pins and is preferred when pin count is limited.

Q6. What is the role of the RS, RW, and EN pins in LCD?

Ans: RS (Register Select): LOW = Command mode, HIGH = Data mode. RW (Read/Write): LOW = Write to LCD, HIGH = Read from LCD (usually kept LOW). EN (Enable): A HIGH-to-LOW pulse latches data into the LCD controller. These three pins control all LCD operations.

Q7. What is the command to clear the LCD display? Give its hex value.

Ans: The command to clear the LCD display is 0x01 (hex). When this command is sent in command mode (RS=0), the LCD clears all characters and returns the cursor to the home position (row 0, column 0).

Q8. What is debouncing in keyboard interfacing? Why is it necessary?

Ans: When a mechanical key is pressed or released, it bounces electrically, causing multiple rapid transitions before settling. Debouncing eliminates these false triggers. It can be done in software (by adding a small delay ~20ms after detecting a key press) or in hardware (using RC filter or Schmitt trigger).

Q9. What are GPIO ports in LPC2148? How do you configure them as input/output?

Ans: LPC2148 has two GPIO ports: Port 0 (P0.0–P0.31) and Port 1 (P1.16–P1.31). IODIR register is used to configure direction: writing 1 sets the pin as OUTPUT, writing 0 sets it as INPUT. IOSET and IOCLR registers are used to set or clear output pins.

Q10. What is PINSEL register? What is its purpose in LPC2148?

Ans: PINSEL registers (PINSEL0, PINSEL1, PINSEL2) are used to select the function of each pin in LPC2148. Each pin has 2 bits in PINSEL: 00 = GPIO, 01 = Primary alternate function (e.g., UART, SPI), 10/11 = Secondary alternate functions. By default, all pins are GPIO (00).

Practical 2: Interfacing LPC2148 with LED Bank to Flash with Timer Delay

Q1. How many timers are present in LPC2148? Name them.

Ans: LPC2148 has two 32-bit timers: Timer0 (T0) and Timer1 (T1). Each timer has a Prescale Register, Match Registers (MR0–MR3), Capture Registers, and a Timer Counter (TC). Both timers can be used for delay generation, PWM, or event counting.

Q2. What is the difference between Timer0 and Timer1 in LPC2148?

Ans: Both Timer0 and Timer1 are functionally identical 32-bit timers. The difference is only in their base register addresses. Timer0 is at base address 0xE0004000 and Timer1 at 0xE0008000. They can operate independently or simultaneously.

Q3. Explain the working of Prescale Register (PR) in LPC2148 timer.

Ans: The Prescale Register (T0PR) defines the value at which the Prescale Counter (PC) resets and increments the Timer Counter (TC). It divides the peripheral clock (PCLK). If $PR = N$, TC increments every $(N+1)$ PCLK cycles, effectively slowing the timer count rate.

Q4. What is a Match Register (MR)? How is it used to generate delays?

Ans: Match Registers (MR0–MR3) are compared continuously with the Timer Counter (TC). When $TC == MR$, a match event occurs. The Match Control Register (MCR) configures the action: interrupt, reset TC, or stop the timer. For delay generation, we set $MR0 = \text{desired_count}$ and wait for the match interrupt or flag.

Q5. What is the function of T0TCR (Timer Control Register)?

Ans: T0TCR controls the timer operation. Bit 0 (Counter Enable): set to 1 to enable the timer, 0 to disable. Bit 1 (Counter Reset): set to 1 to reset TC and PC to zero. To start a timer: write 0x01. To reset: write 0x02, then 0x01 to restart.

Q6. How do you calculate the time delay using LPC2148 timers?

Ans: Time delay = $(MR0 + 1) / PCLK$. For example, if $PCLK = 15 \text{ MHz}$ and we want 1 ms delay: $MR0 = 15000 - 1 = 14999$. Set $PR = 0$ (no prescaling), load $MR0 = 14999$, enable timer, and wait until TC matches $MR0$.

Q7. What is the difference between polling and interrupt-based timer delay?

Ans: In polling mode, the CPU continuously checks the match flag in a while loop (busy-wait), wasting CPU cycles. In interrupt mode, the timer fires an ISR (Interrupt Service Routine) when a match occurs, freeing the CPU for other tasks. Interrupt mode is more efficient for real-time applications.

Q8. How many LEDs did you interface and on which GPIO port?

Ans: Typically, 8 LEDs are interfaced on Port 0 (P0.0–P0.7). The IODIR0 register is configured to set these pins as output. IOSET0 is used to turn LEDs ON and IOCLR0 to turn them OFF. A timer delay is used between ON and OFF states to create a flashing effect.

Q9. What is the purpose of the IODIR register in LPC2148?

Ans: IODIR (IO Direction Register) sets each GPIO pin as input or output. Writing a 1 to a bit position makes that pin an OUTPUT; writing 0 makes it an INPUT. For example, IODIR0 = 0x000000FF configures P0.0–P0.7 as outputs (for LED interfacing).

Q10. What is the clock source for timers in LPC2148? What is PCLK?

Ans: PCLK (Peripheral Clock) is the clock source for LPC2148 timers. PCLK is derived from CCLK (CPU clock) divided by the APBDIV (APB divider) register. If CCLK = 60 MHz and APBDIV = 4, then PCLK = 15 MHz. Most peripherals including timers, UART, and ADC run on PCLK.

Practical 3: Using Built-in ADC of LPC2148 for Displaying Values Without Interrupt

Q1. What type of ADC is built into LPC2148? What is its resolution?

Ans: LPC2148 has a 10-bit successive approximation ADC (SAR ADC). It can convert analog input voltages to a 10-bit digital value (0 to 1023). It has 8 channels (AD0.0–AD0.7) and can operate at a maximum conversion rate of 2.44 MHz clock (max 400 ksp/s).

Q2. How many ADC channels are available in LPC2148?

Ans: LPC2148 has 8 ADC input channels: AD0.0 through AD0.7. These channels are multiplexed into a single ADC core. The desired channel is selected using bits [2:0] (SEL field) of the AD0CR register. All channels share the same 10-bit SAR ADC hardware.

Q3. What is the reference voltage for ADC in LPC2148?

Ans: The ADC reference voltage in LPC2148 is VDDA (Analog supply voltage), typically 3.3 V. The analog input voltage must be between 0 V and VDDA. The digital output is proportional: Digital Value = $(V_{in} / V_{ref}) \times 1023$ for a 10-bit ADC.

Q4. Explain the formula to convert ADC digital output to analog voltage.

Ans: Analog Voltage = $(ADC_Result / 1023) \times V_{ref}$. For example, if ADC result = 512 and $V_{ref} = 3.3$ V, then Voltage = $(512/1023) \times 3.3 = 1.65$ V (approximately). The result is a 10-bit value stored in bits [15:6] of the AD0DR register.

Q5. What is the role of AD0CR (A/D Control Register)?

Ans: AD0CR configures the ADC operation: Bits [7:0] SEL = channel selection, Bits [15:8] CLKDIV = clock divider (ADC clock = $PCLK/(CLKDIV+1)$, must be ≤ 4.5 MHz), Bit 16 BURST = 1 for continuous conversion, Bits [26:24] START = conversion start mode (001 = start now). Set PDN bit [21] = 1 to power up ADC.

Q6. What is polling mode in ADC? How does it differ from interrupt-driven ADC?

Ans: In polling mode, after starting a conversion, the CPU continuously reads the DONE bit (bit 31) of AD0DR register in a loop until it becomes 1, then reads the result. In interrupt mode, an ISR is triggered when conversion completes, allowing the CPU to perform other tasks. Polling wastes CPU time but is simpler to implement.

Q7. What is the significance of the DONE bit in AD0DR?

Ans: The DONE bit (bit 31) in the A/D Data Register (AD0DR) is set to 1 by hardware when the ADC conversion is complete. The ADC result (10-bit value) is available in bits [15:6] of AD0DR. Reading AD0DR clears the DONE bit automatically. The program must wait for DONE=1 before reading the result.

Q8. What is the maximum sampling frequency supported by LPC2148 ADC?

Ans: The ADC clock in LPC2148 must not exceed 4.5 MHz (as per datasheet). Since each conversion takes 11 ADC clock cycles, maximum sampling rate = $4.5 \text{ MHz} / 11 = \sim 409 \text{ ksps}$ (kilo samples per second). In practice, PCLK is divided appropriately to keep ADC clock under 4.5 MHz.

Q9. Which pins of LPC2148 are used as ADC input? How are they configured?

Ans: ADC channels AD0.0–AD0.7 correspond to pins P0.28, P0.29, P0.30, P0.25, P0.10, P0.11, P0.12, P0.13 respectively. To use them as ADC, the PINSEL register must be configured to select the ADC alternate function (e.g., for AD0.0 on P0.28, set PINSEL1 bits [25:24] = 01).

Q10. What are the applications of ADC in embedded systems?

Ans: ADC is used in: temperature sensing (using thermistors/LM35), light intensity measurement (LDR), battery voltage monitoring, audio processing (microphone input), pressure/force sensing, potentiometer-based user input, industrial sensor interfacing (4–20 mA current loops), and medical devices (ECG, blood pressure sensors).

Practical 4: Interfacing GPS with LPC2148 for Finding Current Location (Latitude & Longitude)

Q1. What is GPS? How does it determine the position of an object?

Ans: GPS (Global Positioning System) is a satellite-based navigation system. It uses signals from at least 4 satellites to calculate 3D position (latitude, longitude, altitude) using trilateration. Each satellite broadcasts its position and timestamp; the receiver calculates the time of flight of signals to determine distance from each satellite.

Q2. What is NMEA protocol? Name the common NMEA sentences.

Ans: NMEA 0183 is a standard serial communication protocol used by GPS devices. Data is sent as ASCII text sentences starting with '\$'. Common sentences: \$GPRMC (Recommended Minimum Data), \$GPGLL (Fix data, altitude), \$GPGSV (Satellites in view), \$GPVTG (Course and speed), \$GPGGA (Geographic position).

Q3. What is the GPRMC sentence? What information does it contain?

Ans: \$GPRMC contains: Time (UTC), Status (A=Active/V=Void), Latitude, N/S indicator, Longitude, E/W indicator, Speed over ground (knots), Course over ground (degrees), Date, Magnetic variation. Example: \$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A

Q4. What is the GPGGA sentence? How is it different from GPRMC?

Ans: \$GPGGA provides GPS fix data including: Time, Latitude, Longitude, Fix quality (0=invalid,1=GPS,2=DGPS), Number of satellites, HDOP, Altitude, and geoid separation. Unlike GPRMC, GPGGA includes altitude and fix quality details but does not include date, speed, or course information.

Q5. What serial communication protocol is used between GPS module and LPC2148?

Ans: GPS modules communicate using UART (Universal Asynchronous Receiver Transmitter) serial protocol. The GPS module transmits NMEA sentences continuously via its TX pin, which connects to the RX pin of LPC2148's UART. Communication is one-directional (GPS to microcontroller) for position data.

Q6. What is the standard baud rate used for GPS module communication?

Ans: The standard baud rate for most GPS modules (e.g., Neo-6M, u-blox) is 9600 bps with 8 data bits, no parity, and 1 stop bit (9600-8N1). Some modules support 4800 or 38400 bps, but 9600 bps is the default and most widely used setting.

Q7. How do you parse NMEA data to extract latitude and longitude?

Ans: After receiving the \$GPRMC sentence, split the string by commas. Field[3] = Latitude (DDMM.MMMM format), Field[4] = N/S, Field[5] = Longitude (DDDMM.MMMM format), Field[6] = E/W. Convert: Degrees = int(value/100); Minutes = value - (Degrees*100); Decimal = Degrees + Minutes/60. Apply N/S, E/W sign.

Q8. Which UART of LPC2148 did you use for GPS interfacing?

Ans: UART0 of LPC2148 is commonly used for GPS interfacing. UART0 TX is on P0.0 and RX is on P0.1. Configure PINSEL0 to enable UART0 function on these pins (bits [1:0] = 01 for TX, bits [3:2] = 01 for RX). Initialize UART0 at 9600 baud rate.

Q9. What is the difference between geographic coordinates in DMS and decimal degrees?

Ans: DMS (Degrees, Minutes, Seconds): e.g., 18°31'30"N. Decimal Degrees: e.g., 18.525°N. Conversion: DD = D + M/60 + S/3600. GPS NMEA data provides coordinates in DDMM.MMMM format (degrees and decimal minutes), which must be further converted to decimal degrees for mapping applications like Google Maps.

Q10. What are the factors that affect GPS accuracy?

Ans: GPS accuracy is affected by: Atmospheric delays (ionosphere, troposphere), Multipath errors (signal reflection off buildings), Satellite geometry (DOP – Dilution of Precision), Number of visible satellites, Receiver quality, Urban canyons blocking signals, and Solar activity. DGPS (Differential GPS) can improve accuracy to below 1 meter.

Practical 5: Interfacing UART of LPC2148 for Serial Communication — Reception and Transmission to Computer

Q1. What is UART? Expand the abbreviation and explain its working.

Ans: UART stands for Universal Asynchronous Receiver Transmitter. It is a serial communication protocol that transmits data bit-by-bit without a shared clock signal (asynchronous). Both devices must agree on the same baud rate. Data is framed with a start bit (LOW), data bits (LSB first), optional parity bit, and stop bit(s) (HIGH).

Q2. How many UARTs are present in LPC2148? Which pins are used for UART0?

Ans: LPC2148 has two UARTs: UART0 and UART1. UART0: TXD0 on P0.0, RXD0 on P0.1. UART1: TXD1 on P0.8, RXD1 on P0.9 (also supports modem control signals). Configure using PINSEL0 register. UART0 is most commonly used for PC communication.

Q3. What is baud rate? How do you configure baud rate in LPC2148?

Ans: Baud rate is the number of signal changes per second (bits per second for UART). To configure: Set DLAB bit in U0LCR = 1, then write divisor values to U0DLL (low byte) and U0DLM (high byte). Divisor = PCLK / (16 x Baud Rate). For 9600 baud with PCLK=15 MHz: Divisor = 15000000/(16x9600) = 97.6 ≈ 98.

Q4. Explain the UART frame format (start bit, data bits, stop bit, parity).

Ans: UART frame: 1 Start bit (always LOW) + 5–8 Data bits (LSB first) + 1 optional Parity bit (even/odd/none) + 1 or 2 Stop bits (always HIGH). Idle line is HIGH. The start bit signals the beginning of transmission; the receiver samples each bit at the center of its period based on the agreed baud rate.

Q5. What is the U0LCR register? What parameters does it configure?

Ans: U0LCR (UART0 Line Control Register) configures the frame format: Bits [1:0] = Word Length (00=5-bit, 11=8-bit), Bit 2 = Stop bits (0=1 stop, 1=2 stop), Bit 3 = Parity Enable, Bits [5:4] = Parity type, Bit 7 = DLAB (Divisor Latch Access Bit). Common setting: U0LCR = 0x83 (8-bit, 1 stop, no parity, DLAB=1).

Q6. What is the difference between UART, USART, and RS232?

Ans: UART: Asynchronous only, no clock line. USART: Universal Synchronous/Asynchronous Receiver Transmitter — supports both synchronous (with clock) and asynchronous modes. RS232: A voltage-level standard for serial communication (±12 V logic levels). UART uses TTL levels (0/3.3V or 0/5V). A MAX232 IC is used to convert TTL UART levels to RS232 levels for PC COM ports.

Q7. What is the role of U0THR and U0RBR registers?

Ans: U0THR (Transmit Holding Register): Writing a byte here places it in the TX FIFO for transmission. U0RBR (Receive Buffer Register): Reading this register retrieves the oldest received byte from the RX FIFO. Both registers share the same address 0xE00C000; U0THR is write-only and U0RBR is read-only (DLAB=0).

Q8. What is the DLAB bit? Why is it important during baud rate configuration?

Ans: DLAB (Divisor Latch Access Bit) is bit 7 of U0LCR. When DLAB=1, the addresses of U0THR/U0RBR are remapped to U0DLL and U0DLM (baud rate divisor registers), allowing baud rate configuration. After setting the baud rate divisor, DLAB must be set back to 0 so U0THR and U0RBR become accessible for data transfer.

Q9. Which software tool did you use on the PC side for serial communication?

Ans: Common tools used for serial communication with LPC2148: PuTTY (open-source, supports Serial/SSH/Telnet), Tera Term, HyperTerminal (older Windows), RealTerm, or Arduino Serial Monitor. Configure the tool with matching baud rate (9600), 8 data bits, no parity, 1 stop bit, and the correct COM port number.

Q10. What is flow control in UART? Difference between hardware and software flow control?

Ans: Flow control prevents buffer overflow when one device is faster than the other. Hardware flow control uses RTS (Request to Send) and CTS (Clear to Send) signal lines — the sender waits for CTS before transmitting. Software flow control (XON/XOFF) uses special ASCII characters (Ctrl+S to pause, Ctrl+Q to resume) sent within the data stream. Hardware flow control is more reliable for high-speed communication.

Practical 6: Interfacing LED and Seven Segment LED using STM32F4xx

Q1. What is STM32F4xx? Which ARM core does it use?

Ans: STM32F4xx is a family of high-performance 32-bit microcontrollers from STMicroelectronics based on the ARM Cortex-M4 core with FPU (Floating Point Unit). The STM32F407 variant runs at up to 168 MHz, has 1 MB Flash, 192 KB RAM, and rich peripherals including ADC, DAC, UART, SPI, I2C, USB, CAN, and Ethernet.

Q2. What is the maximum operating frequency of STM32F407?

Ans: The STM32F407 can operate at a maximum CPU frequency of 168 MHz. This is achieved using the on-chip PLL (Phase-Locked Loop) fed by an external 8 MHz HSE (High Speed External) crystal. The system clock (SYSCLK) is configured via the RCC (Reset and Clock Control) registers or using HAL_RCC_ClockConfig().

Q3. How do you enable a GPIO clock in STM32F4? Which register is used?

Ans: In STM32F4, GPIO clocks are enabled via the RCC_AHB1ENR register. Each bit enables the clock for a specific GPIO port. Example: `RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN` enables GPIOA clock. Using HAL: `__HAL_RCC_GPIOA_CLK_ENABLE()`. Without enabling the clock, the GPIO registers are inaccessible and the peripheral won't work.

Q4. What is the difference between GPIO_MODE_OUTPUT_PP and GPIO_MODE_OUTPUT_OD?

Ans: GPIO_MODE_OUTPUT_PP (Push-Pull): The output can actively drive HIGH (via PMOS) or LOW (via NMOS). Suitable for directly driving LEDs. GPIO_MODE_OUTPUT_OD (Open-Drain): The output can only pull LOW (via NMOS); for HIGH, it relies on an external pull-up resistor. Open-drain is used for I2C bus and wired-AND configurations.

Q5. How is a seven-segment display common anode different from common cathode?

Ans: Common Cathode (CC): All cathodes of LEDs are connected to GND. Segments are turned ON by applying HIGH (1) to the segment pin. Common Anode (CA): All anodes of LEDs are connected to VCC. Segments are turned ON by applying LOW (0) to the segment pin. The segment codes (digit patterns) are inverted between CC and CA types.

Q6. Write the segment codes for digits 0–9 for a common cathode seven-segment display.

Ans: Segment order: gfedcba (bit7=dp). 0=0x3F, 1=0x06, 2=0x5B, 3=0x4F, 4=0x66, 5=0x6D, 6=0x7D, 7=0x07, 8=0x7F, 9=0x6F. These codes energize the correct segments (a–g) to display each digit. For common anode, use the bitwise complement of these values.

Q7. What is HAL library in STM32? What is its advantage over direct register programming?

Ans: HAL (Hardware Abstraction Layer) is a set of C driver APIs provided by STMicroelectronics (via STM32CubeIDE/CubeMX). It abstracts hardware registers into easy-to-use functions like HAL_GPIO_WritePin(), HAL_UART_Transmit(). Advantages: portability across STM32 families, faster development, auto-generated initialization code. Disadvantage: slightly larger code size and lower execution speed compared to direct register access.

Q8. What is the role of RCC (Reset and Clock Control) in STM32F4?

Ans: RCC manages all clock sources, PLLs, clock distribution to peripherals, and system resets. It controls: HSI (16 MHz internal RC), HSE (external crystal), PLL configuration, bus clock dividers (AHB, APB1, APB2), and peripheral clock enables. All peripherals must have their clock enabled via RCC before use, otherwise they remain in reset state.

Q9. How do you use HAL_GPIO_WritePin() and HAL_GPIO_TogglePin() functions?

Ans: HAL_GPIO_WritePin(GPIOx, GPIO_Pin, PinState): Sets a pin HIGH or LOW. Example: HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET) sets PA5 HIGH. HAL_GPIO_TogglePin(GPIOx, GPIO_Pin): Inverts the current state of the pin. Example: HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5) toggles PA5 — useful for LED blinking in a loop with HAL_Delay().

Q10. What is the difference between STM32F4 and LPC2148 in terms of architecture?

Ans: STM32F407 uses ARM Cortex-M4 (168 MHz, FPU, DSP instructions, 1 MB Flash, 192 KB RAM). LPC2148 uses ARM7TDMI-S (60 MHz, no FPU, 512 KB Flash, 40 KB RAM). STM32F4 has more peripherals, higher speed, and a more modern architecture. LPC2148 is simpler and widely used in education. HAL library makes STM32 easier to program; LPC2148 uses direct register programming.

Practical 7: Write Embedded C Program to Transmit a Character from Keyboard Using On-chip UART for STM32F4xx

Q1. Which UART peripherals are available in STM32F407? How many are there?

Ans: STM32F407 has 6 UART/USART peripherals: USART1, USART2, USART3 (full USART with synchronous support), UART4, UART5 (async only, no synchronous/smartcard/IrDA/LIN), and USART6. USART1 and USART6 are on the faster APB2 bus (max 84 MHz); others are on APB1 bus (max 42 MHz).

Q2. How do you initialize UART in STM32F4 using HAL library?

Ans: 1) Enable GPIO and UART clocks via RCC. 2) Configure GPIO pins as Alternate Function (AF) mode with the correct AF number. 3) Fill UART_HandleTypeDef structure: Instance (e.g., USART2), BaudRate, WordLength (UART_WORDLENGTH_8B), StopBits, Parity, Mode (TX+RX). 4) Call HAL_UART_Init(&huart2;). STM32CubeMX can auto-generate this initialization code.

Q3. What is the function HAL_UART_Transmit()? What parameters does it take?

Ans: HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout). Parameters: huart = pointer to UART handle, pData = pointer to data buffer, Size = number of bytes to send, Timeout = max wait time in ms (HAL_MAX_DELAY for infinite). Returns HAL_OK on success, HAL_TIMEOUT if time expires.

Q4. What is HAL_UART_Receive()? What is the timeout parameter used for?

Ans: HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout) receives 'Size' bytes into the buffer pointed by pData. The Timeout parameter specifies the maximum time (in ms) to wait for data. If data doesn't arrive within Timeout ms, the function returns HAL_TIMEOUT. Use HAL_MAX_DELAY to wait indefinitely.

Q5. What is the difference between blocking and non-blocking UART modes?

Ans: Blocking (Polling): HAL_UART_Transmit/Receive — CPU waits until transfer completes. Simple but inefficient. Non-Blocking Interrupt: HAL_UART_Transmit_IT/Receive_IT — transfer starts, CPU is free, an interrupt fires when done (callback). Non-Blocking DMA: HAL_UART_Transmit_DMA/Receive_DMA — DMA handles data transfer without CPU involvement, most efficient for large data transfers.

Q6. How do you configure baud rate, word length, stop bits, and parity in STM32F4?

Ans: Configure via `UART_HandleTypeDef`: `.BaudRate = 9600` (or desired rate), `.WordLength = UART_WORDLENGTH_8B`, `.StopBits = UART_STOPBITS_1`, `.Parity = UART_PARITY_NONE`, `.Mode = UART_MODE_TX_RX`, `.HwFlowCtl = UART_HWCONTROL_NONE`. Internally, HAL calculates and loads the BRR (Baud Rate Register) based on the APB clock and desired baud rate.

Q7. What is the USART_SR register? Which bits indicate TX/RX readiness?

Ans: USART_SR (Status Register) contains: Bit 5 (RXNE - Receive Not Empty): set when received data is ready to be read from DR. Bit 6 (TC - Transmission Complete): set when last byte transmission is complete. Bit 7 (TXE - Transmit Data Register Empty): set when TDR is empty and ready for new data. Polling these bits allows software UART operation without HAL.

Q8. How is STM32F4 UART different from LPC2148 UART in register configuration?

Ans: LPC2148 UART: Uses U0LCR, U0DLL, U0DLM, U0THR, U0RBR, U0LSR registers. DLAB bit required for baud rate setup. Direct register access. STM32F4 UART: Uses USART_BRR, USART_CR1, USART_CR2, USART_DR registers. BRR is directly loaded with divider value. HAL library provides higher abstraction. STM32F4 UART supports more modes (IrDA, SmartCard, LIN, synchronous).

Q9. What is DMA? How can DMA be used for UART in STM32F4?

Ans: DMA (Direct Memory Access) transfers data between memory and peripherals without CPU involvement. For UART DMA in STM32F4: use `HAL_UART_Transmit_DMA()` or `HAL_UART_Receive_DMA()`. DMA streams are configured with source/destination addresses, data width, and transfer count. CPU starts the DMA, then handles other tasks; a DMA transfer complete callback signals when done. Ideal for large or continuous data streams.

Q10. What software tool did you use to verify serial transmission from STM32F4 to PC?

Ans: PuTTY or Tera Term is used on the PC side. A USB-to-UART bridge (e.g., CP2102, CH340, or FT232 module) converts STM32 UART TTL levels to USB for PC connection. Configure the terminal with matching settings: baud rate (e.g., 9600), 8N1, and the correct COM port. Characters typed on the PC keyboard are transmitted to STM32 and echoed back for verification.

Practical 8: Write Embedded C Program to Perform On-chip ADC Implementation with STM32F4xx

Q1. What type of ADC is used in STM32F407? What is its resolution?

Ans: STM32F407 has three 12-bit SAR (Successive Approximation Register) ADCs: ADC1, ADC2, ADC3. Resolution is configurable: 12-bit (4096 levels), 10-bit, 8-bit, or 6-bit. Default is 12-bit. Maximum sampling rate is 2.4 Msps (mega samples per second) at 12-bit resolution with 36 MHz ADC clock.

Q2. How many ADC modules and channels are available in STM32F407?

Ans: STM32F407 has 3 ADC modules (ADC1, ADC2, ADC3). Each has up to 19 channels: 16 external multiplexed channels (IN0–IN15) and 3 internal channels (temperature sensor, VREFINT, VBAT). ADC1 and ADC2 can operate in dual mode for simultaneous sampling. Total unique external ADC pins: 16 channels shared across the three ADCs.

Q3. What is single conversion mode vs. continuous conversion mode in STM32 ADC?

Ans: Single Conversion Mode: ADC performs one conversion when triggered, then stops. Must be retriggered for each conversion. Set `CONT=0` in `ADC_CR2`. Continuous Conversion Mode: ADC converts repeatedly without stopping after the first trigger. Set `CONT=1`. Useful for real-time monitoring. Scan mode can be combined with either mode to convert multiple channels sequentially.

Q4. What is the reference voltage (Vref) for ADC in STM32F407?

Ans: STM32F407 ADC uses `VREF+` as the positive reference voltage, typically 3.3 V (connected to `VDDA`). The negative reference is `VREF-` (connected to `VSSA = GND`). Digital value = $(V_{in} / VREF+) \times 4095$ for 12-bit. An internal band-gap reference (`VREFINT` ≈ 1.21 V) is also available as an internal channel for calibration purposes.

Q5. How do you start ADC conversion using HAL? Which function is used?

Ans: Steps: 1) Initialize ADC with `HAL_ADC_Init()`. 2) Configure channel with `HAL_ADC_ConfigChannel()`. 3) Start conversion: `HAL_ADC_Start(&hadc1;)`. 4) Wait for completion: `HAL_ADC_PollForConversion(&hadc1;, HAL_MAX_DELAY)`. 5) Read result: `HAL_ADC_GetValue(&hadc1;)`. 6) Stop: `HAL_ADC_Stop(&hadc1;)`. For interrupt mode: use `HAL_ADC_Start_IT()` and implement `HAL_ADC_ConvCpltCallback()`.

Q6. What is the formula to convert ADC raw value to voltage in STM32F4?

Ans: Voltage = $(ADC_Raw_Value / 4095.0) \times VREF$. For 12-bit ADC with `VREF = 3.3 V`: Voltage = $(Raw / 4095.0) \times 3.3$. Example: if raw value = 2048, Voltage = $(2048/4095) \times 3.3 = 1.65$ V. For display, multiply by 1000 for mV to avoid floating point and use integer arithmetic in embedded code.

Q7. What is oversampling in ADC? What is its benefit?

Ans: Oversampling means taking multiple ADC samples and averaging them to improve effective resolution. For example, averaging 16 samples of 12-bit ADC gives ~14-bit effective resolution (each doubling of samples adds 0.5 bits). STM32 hardware oversampling (available in newer STM32 series) automates this. It reduces noise and improves measurement accuracy at the cost of slower throughput.

Q8. What is the ADC clock source in STM32F4? How does it affect sampling rate?

Ans: The ADC clock in STM32F4 is derived from `APB2` clock divided by a prescaler (2, 4, 6, or 8), set in `ADC_CCR` register. Maximum ADC clock is 36 MHz. ADC conversion time = Sampling cycles + 12 cycles (for 12-bit). Total time = $(Sampling_cycles + 12) / ADC_CLK$. Lower ADC clock = slower but more accurate conversion; higher clock = faster but may reduce accuracy.

Q9. Compare the ADC of LPC2148 and STM32F407.

Ans: LPC2148 ADC: 10-bit resolution, 8 channels, max 400 ksp/s, single ADC module. STM32F407 ADC: 12-bit resolution (configurable), 16 external channels, 3 ADC modules, max 2.4 Msps, supports dual/triple modes, DMA, hardware oversampling, internal temperature sensor channel, and injected channel group for high-priority conversions. STM32F407 is significantly more powerful.

Q10. What are the real-world applications of ADC interfacing in embedded systems?

Ans: ADC is used in: Industrial automation (pressure, temperature, flow sensors), Medical devices (ECG signal acquisition, pulse oximetry, blood pressure), Audio systems (microphone digitization), Automotive (throttle position, oxygen sensors), Power electronics (current/voltage monitoring), Consumer electronics (battery level indicators, touchscreens), IoT devices (environmental sensors — humidity, CO2), and Robotics (proximity, force sensing).

Q1. What is MPU6050? What sensors are integrated inside it?

Ans: MPU6050 is a 6-axis IMU (Inertial Measurement Unit) from InvenSense. It integrates a 3-axis MEMS accelerometer and a 3-axis MEMS gyroscope in a single IC. It also has an embedded temperature sensor and a DMP (Digital Motion Processor) for onboard sensor fusion. It communicates via I2C (or SPI in MPU6000 variant).

Q2. What communication protocol does MPU6050 use? What is its I2C address?

Ans: MPU6050 uses I2C communication. The I2C address is 0x68 (7-bit) when AD0 pin is connected to GND, or 0x69 when AD0 is connected to VCC. This allows two MPU6050 devices on the same I2C bus. The I2C bus speed can be 100 kHz (standard) or 400 kHz (fast mode).

Q3. Explain the working principle of a MEMS accelerometer.

Ans: A MEMS accelerometer uses a tiny proof mass suspended by springs on a silicon chip. When acceleration is applied, the proof mass deflects, changing the capacitance between the mass and fixed electrodes. This capacitance change is converted to voltage, which is then digitized by an ADC. MPU6050 measures acceleration in 3 axes (X, Y, Z) in units of g (gravity).

Q4. What is a gyroscope? How does it measure angular velocity?

Ans: A MEMS gyroscope uses the Coriolis effect. A vibrating mass is driven to oscillate. When angular rotation is applied, Coriolis force deflects the mass perpendicular to its vibration, changing capacitance. This is measured and converted to angular velocity in degrees per second (°/s). MPU6050 measures rotation about X, Y, Z axes.

Q5. What is the DMP (Digital Motion Processor) in MPU6050?

Ans: The DMP is an on-chip processor in MPU6050 that performs sensor fusion algorithms (combining accelerometer and gyroscope data) to compute quaternions, Euler angles (roll, pitch, yaw), and tap detection. Using DMP offloads computation from the main MCU, reduces host CPU load, and provides more stable orientation estimates by compensating for gyroscope drift using accelerometer data.

Q6. What are the WHO_AM_I, ACCEL_XOUT_H, and GYRO_XOUT_H registers?

Ans: WHO_AM_I (register 0x75): Returns 0x68 — used to verify MPU6050 is communicating correctly. ACCEL_XOUT_H (0x3B): High byte of raw X-axis accelerometer data (16-bit two's complement). Similarly ACCEL_XOUT_L (0x3C) for low byte. GYRO_XOUT_H (0x43): High byte of raw X-axis gyroscope data. Full 16-bit signed values are read by combining high and low bytes.

Q7. How do you initialize I2C in STM32F4 using HAL library?

Ans: 1) Enable GPIO and I2C peripheral clocks via RCC. 2) Configure SCL and SDA pins as Alternate Function Open-Drain with pull-up. 3) Fill I2C_HandleTypeDef: Instance (I2C1), ClockSpeed (100000 or 400000 Hz), DutyCycle, OwnAddress1, AddressingMode (7-bit). 4) Call HAL_I2C_Init(&hi2c1;). STM32CubeMX auto-generates this configuration.

Q8. What is HAL_I2C_Mem_Read()? How is it used to read data from MPU6050?

Ans: HAL_I2C_Mem_Read(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout). Example: HAL_I2C_Mem_Read(&hi2c1, 0xD0, 0x3B, I2C_MEMADD_SIZE_8BIT, buffer, 14, 100) reads 14 bytes starting from register 0x3B (all accel, temp, gyro data) from MPU6050 at address 0x68 (0xD0 = 0x68<<1).

Q9. How do you convert raw accelerometer data to 'g' values and gyroscope data to °/s?

Ans: For accelerometer: Sensitivity depends on FS_SEL setting. At $\pm 2g$: sensitivity = 16384 LSB/g. Acceleration (g) = Raw_Value / 16384.0. For gyroscope: At $\pm 250^\circ/\text{s}$ (default): sensitivity = 131 LSB/ $^\circ/\text{s}$. Angular rate = Raw_Value / 131.0. Two bytes (high and low) are combined: value = (int16_t)((high_byte << 8) | low_byte) to get signed 16-bit value.

Q10. What are the applications of MPU6050 in real-world embedded systems?

Ans: MPU6050 is used in: Drones/quadcopters (flight stabilization using PID control), Robotics (balance bots, robotic arms), Smartphones (screen auto-rotation, step counting), Gaming controllers (motion sensing), Augmented/Virtual Reality headsets, Vehicle stability systems, Gesture recognition systems, Prosthetic limbs (movement control), and Wearable fitness trackers (activity monitoring).

Practical 10: Interfacing Ultrasonic Sensor HC-SR04 with STM32F4xx

Q1. What is HC-SR04? What is the principle of operation of an ultrasonic sensor?

Ans: HC-SR04 is a popular ultrasonic distance measurement module. It uses the principle of echolocation (like bats). The module emits a burst of 40 kHz ultrasonic sound pulses, which travel through air, reflect off an object, and return to the sensor. Distance is calculated by measuring the time between transmission and reception of the echo.

Q2. What is the operating voltage and measuring range of HC-SR04?

Ans: HC-SR04 operates at 5V DC. Measuring range: 2 cm to 400 cm (4 meters). Accuracy: ± 3 mm. Measuring angle: 15° . Trigger pulse duration: 10 μs minimum. The module has 4 pins: VCC (5V), GND, TRIG (input), and ECHO (output). Note: ECHO pin outputs 5V; a voltage divider may be needed for 3.3V MCUs like STM32F4.

Q3. Explain the Trigger and Echo pins of HC-SR04 and their operation.

Ans: TRIG pin: MCU sends a 10 μs HIGH pulse to trigger a measurement. The HC-SR04 then sends 8 pulses of 40 kHz ultrasonic sound. ECHO pin: Goes HIGH after transmission starts and returns LOW when the echo is received. The duration of the HIGH pulse on ECHO is proportional to the distance. MCU measures this pulse width using a timer.

Q4. What is the formula to calculate distance using HC-SR04?

Ans: Distance = (Echo_Pulse_Width_in_ μs x Speed_of_Sound) / 2. Speed of sound at 20°C = 343 m/s = 0.0343 cm/ μs . So: Distance (cm) = Echo_time_ μs x 0.0343 / 2 = Echo_time_ μs / 58. Example: if echo pulse = 1160 μs , distance = 1160/58 = 20 cm. The division by 2 accounts for the round trip (to object and back).

Q5. What timer feature of STM32F4 is used to measure the Echo pulse width?

Ans: Input Capture mode of STM32F4 timers is used to measure the Echo pulse width. The timer captures the counter value at both the rising edge (echo start) and falling edge (echo end) of the ECHO pin. The difference between the two captured values gives the pulse width in timer ticks, which is then converted to microseconds using the timer's clock frequency.

Q6. What is Input Capture mode in STM32F4 timer? How is it used for HC-SR04?

Ans: In Input Capture mode, the timer captures the value of its counter register (CNT) when a specified edge (rising/falling) occurs on the input pin. For HC-SR04: Configure timer in Input Capture. Capture on rising edge $\rightarrow$ record time T1. Switch to falling edge capture $\rightarrow$ record time T2. Echo pulse width = T2 - T1 (in timer ticks). Convert to μs : width_ μs = (T2-T1) / (timer_clock_MHz).

Q7. What is the speed of sound used in HC-SR04 distance calculation?

Ans: The standard speed of sound used is 343 m/s (at 20°C, dry air) = 34300 cm/s = 0.0343 cm/μs. This gives the formula: Distance = time(μs) / 58 cm. Note: speed of sound varies with temperature: $v = 331.4 + 0.606 \times T(^{\circ}\text{C})$ m/s. For precision applications, temperature compensation should be applied. HC-SR04 does not compensate for temperature.

Q8. What are the limitations of HC-SR04 sensor?

Ans: Limitations: 1) Minimum range of 2 cm (dead zone), 2) Maximum range ~4 m, 3) Works poorly on soft/angled/curved surfaces (sound absorption/reflection issues), 4) Narrow effective angle (15°), 5) Susceptible to interference from other ultrasonic sources, 6) Inaccurate on very small objects, 7) Temperature-dependent accuracy, 8) Cannot detect transparent objects (glass) well, 9) Slow update rate (~50ms between readings recommended).

Q9. How do you generate a 10 microsecond trigger pulse using STM32F4?

Ans: Method 1 (HAL_Delay): Set TRIG pin HIGH, call delay_us(10), set TRIG pin LOW. HAL_Delay() has 1ms resolution, so a custom microsecond delay using a timer is needed. Method 2 (Timer): Configure a timer with 1 μs tick (prescaler = CPU_clock/1MHz - 1), set counter, wait until CNT >= 10, then pull TRIG LOW. Method 3: Use DWT (Data Watchpoint and Trace) cycle counter for μs delay.

Q10. Name two alternative sensors to HC-SR04 and compare their features.

Ans: 1) JSN-SR04T: Waterproof ultrasonic sensor, same operating principle as HC-SR04, range 20 cm–600 cm, suitable for outdoor/liquid level measurement. 2) VL53L0X (ToF Laser): Uses Time-of-Flight laser (LIDAR), range 0–2 m, much faster (up to 50 Hz), more accurate (±1 mm), unaffected by surface color/material, smaller package, I2C interface. ToF sensors are preferred for robotics and drones; HC-SR04 is simpler and cheaper for basic applications.

Note: Study the register-level details for LPC2148 and HAL function parameters for STM32F4. Understand the working principle of each sensor/peripheral. Be ready to draw circuit diagrams and explain code logic.

Best of Luck for your Oral Examination! — Sinhgad College of Engineering, Pune - 41